Counting the Mask: When Does Pruning Pay in Quantized Small Language Models?

A. Yaacoub

[Affiliation to be confirmed]

Abstract. Pruning and quantization are routinely stacked to compress language models, and compression rates that ignore the storage of the sparsity mask make the combination look better than it is. Inside one Hessian-compensated sweep, we compare joint, prune-then-quantize and quantize-then-prune compression of SmolLM2 and Qwen2.5 models from 135M to 0.5B parameters. With the mask counted, 25% pruning with 4-bit weights costs as much as plain 4-bit quantization and only ties it, while 50% pruning with 4-bit weights beats plain 3-bit quantization at 3.0 bits per weight on 2 of 3 models; below 3 bits, plain quantization collapses. Quantizing first is the worst order in most settings, also where a weaker-first rule predicts it to be best, and neither a stale quantization grid nor a mask chosen on rounded weights explains why. We evaluate with KL divergence, answer flips and full benchmark splits.

1 Introduction

Pruning and quantization both have accurate one-shot post-training versions for language models, SparseGPT [1] and GPTQ [2], cast in one optimal-brain-surgeon (OBS) framework [3], and they are routinely stacked. In earlier work [4] we compared stacked configurations at equal theoretical compression rate (TCr), which counts only the stored values. An unstructured sparse matrix must also store where its non-zeros are: with a 1-bit mask, 25% pruning with 4-bit weights costs 4.0 bits per weight, as much as plain 4-bit, not 3.0. SparseGPT stores such a bitmask and reports 50% sparsity with 4-bit weights to be more accurate than 3-bit quantization at the same memory, for OPT models from 2.7B parameters up [1]; on small models, mostly of 1B to 3.8B parameters, quantization preserves quality better than pruning [5].

The order of the two steps is open too. Harma et al. [6] prove that sparsity and max-scaled quantization are non-orthogonal and that sparsity should come first, for magnitude pruning without compensation; for SparseGPT and Wanda they report a much smaller order effect, measured with 6- to 8-bit formats. Kim et al. [7] propose to apply the weaker perturbation first, in sequential pipelines that also quantize activations and the KV cache. Bouriel et al. [8] combine magnitude pruning and bitsandbytes quantization on models of about 1B parameters; for 4-bit hybrids at 20% sparsity the order changes the interaction by at most 1–2 points. None of these uses a joint sweep, and each uses magnitude pruning or quantizes activations. We ask the question inside the one-shot, Hessian-compensated, weight-only pipeline used in practice [1, 9], on SmolLM2 [10] and Qwen2.5 [11] models compressed by one sweep in which only the order differs, and count storage as SparseGPT does. We contribute a storage

frontier with the mask counted, an exact free-zero effect, a same-Hessian test of a layer-error decomposition, a test of two order rules, a negative result on the cause of the order effect, and an evaluation with KL divergence and answer flips [12].

2 One sweep, three orders

For a linear layer with weights W and calibration inputs X, post-training compression minimizes $\|WX - \widehat{W}X\|_F^2$ under a sparsity or precision constraint; with $H = 2XX^\top$ and $\Delta = W - \widehat{W}$ the objective is $\mathrm{tr}(\Delta H \Delta^\top)$, and we report the normalized layer error $E = \mathrm{tr}(\Delta H \Delta^\top)/\mathrm{tr}(WHW^\top)$. OBS-type methods compress the columns in order, compensating each column's error on the remaining ones through H^{-1}. SparseGPT zeroes, per block of 128 columns, the weights of smallest saliency $w^2/[H^{-1}]_{jj}^2$; GPTQ rounds to a min–max asymmetric grid fitted per row and group of 128 columns. One sweep implements three combinations. *Joint* [3, 1]: in each block the mask is selected first, the grid is fitted on the kept weights, and every column is pruned and quantized before compensation. $P{\to}Q$: a pruning sweep returns W_P and its mask M; a quantization sweep keeps M fixed, so pruned weights stay zero while their error is still compensated. $Q{\to}P$: a quantization sweep returns W_Q and its grid G; a pruning sweep selects the mask and projects every compensated weight back onto G. Joint and P→Q are *mask-first*, Q→P is *grid-first*. Layers are compressed sequentially, each calibrated on the outputs of the already compressed ones.

3 Counting storage and interaction

Storage. For sparsity s and bit-width b, $\mathrm{TCr} = 1 - \frac{b}{16}(1-s)$ [4] counts $b(1-s)$ bits per weight, the stored values only. With one mask bit per weight, as SparseGPT's comparison implies [1], a layer costs $B(s,b) = b(1-s) + \mathbf{1}_{s>0} + (16+b)/128$ bits per weight; the last term, a 16-bit scale and a b-bit zero point per group of 128 weights (0.156 bit at $b = 4$, 0.148 at $b = 3$), differs by at most 1/128 bit between the configurations we compare, so we leave it out. The binary entropy $H_2(s)$ is the cost of an ideally coded i.i.d. mask (1 bit at $s = 50\%$, 0.81 at 25%), and entropy coding of the stored symbols, zeros included (symbol entropy), would also make the zeros of plain rounding free. The three orders of a cell (s, b) cost the same; across cells we compare at equal B: 25%+4-bit with plain 4-bit at 4.0 bits, 50%+4-bit with plain 3-bit at 3.0 bits.

Interaction. With layer errors E_P, E_Q and E_{PQ} of pruning only, quantization only and a combination, $I_E = E_{PQ} - (E_P + E_Q)$ is negative when the combination is sub-additive. Write $\langle A, B\rangle_H = \mathrm{tr}(AHB^\top)$ and $\|A\|_H^2 = \langle A, A\rangle_H$, and take a mask M and a fixed grid $q(\cdot)$ without compensation. With $P = W \odot M$ the pruned weights and $R = W - q(W) = R_M + R_{\bar{M}}$ the rounding residual on pruned and kept entries, $\Delta_P = P$, $\Delta_Q = R$, $\Delta_{PQ} = P + R_{\bar{M}}$ and, up to the

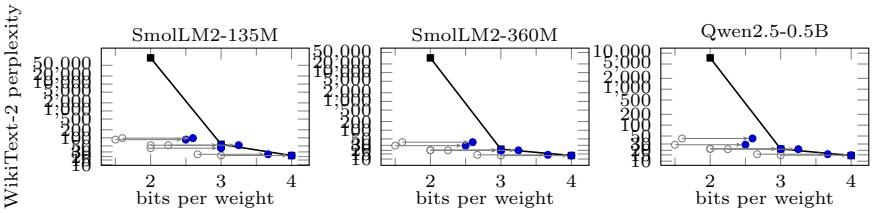


Fig. 1: Counting the mask moves every sparse configuration one bit to the right: plain quantization (black), and pruning + quantization (best order per cell) placed by TCr (gray) and with a 1-bit mask (blue).

common normalization,

$$I_E \;\propto\; \underbrace{-\|R_M\|_H^2}_{\text{absorption} \le 0} \;+\; \underbrace{2\langle P, R_{\bar{M}}\rangle_H - 2\langle R_M, R_{\bar{M}}\rangle_H}_{\text{cross terms}}.$$

Rounding errors on pruned weights disappear, and the cross terms vanish for a diagonal H, since P, R_M and $R_{\bar{M}}$ have disjoint supports within each row. The guarantee $E_{PQ} \le E_P + E_Q$ thus needs no compensation, a fixed grid and a diagonal H. Language-model inputs are correlated, so the cross terms can take either sign, and in sequential runs E_P, E_Q and E_{PQ} come from different Hessians from the second block on; we therefore test sub-additivity with one shared Hessian per layer.

4 Experimental setup

We compress all linear layers of the decoder blocks of SmolLM2-135M and -360M and Qwen2.5-0.5B. Calibration uses 128 random windows of 2048 tokens from the first validation shard of C4 [13], a damping of $0.01\,\overline{\mathrm{diag}(H)}$, and blocks and groups of 128 columns. The grid (E1) crosses $s \in \{25, 33.3, 50\}\%$ with $b \in \{4, 3\}$ in the three orders, plus each method alone ($b = 8$ included); E4 adds 2-bit, 60% and 60%+4-bit. E2 recomputes every variant of a layer with one dense-model Hessian, for every second block; E3 repeats its arms as full runs. All runs use fp32 on NVIDIA L4 GPUs, TF32 off. We report WikiText-2 [14] and C4 perplexity, the token-level KL divergence from the dense to the compressed model on WikiText-2, and zero-shot accuracy with lm-eval [15] on the full splits of nine tasks, logged per item (ARC-Easy and -Challenge, HellaSwag, PIQA, WinoGrande, OpenBookQA, BoolQ, SciQ, LAMBADA). Flips are the items whose correctness differs from the dense model, pooled over the tasks [12]; intervals are paired bootstraps (10000 resamples), and orders are compared item by item with exact McNemar tests. Code and results: `github.com/martin-cousseau/esann2027-compose`; models: `huggingface.co/martincousseau/esann2027-compose-v2`; runs: `wandb.ai/martin-css/esann2027-compose-v2`.

		4.0 bits per weight				3.0 bits per weight			
		4-bit	25% + 4-bit			3-bit	50% + 4-bit		
Model			joint	P→Q	Q→P		joint	P→Q	Q→P
SmolLM2-135M	PPL	**21.94**	23.50	21.99	24.35	57.55	44.47	**41.20**	47.33
(dense 15.42)	KL	**0.346**	0.414	0.348	0.449	1.273	1.043	**0.966**	1.112
	Flips (%)	14.4	14.5	**13.0**	15.5	27.4	22.8	**22.0**	**22.0**
SmolLM2-360M	PPL	13.27	**13.20**	13.34	14.38	21.97	**20.48**	21.29	23.63
(dense 11.46)	KL	0.143	**0.139**	0.146	0.222	0.650	**0.589**	0.631	0.727
	Flips (%)	**10.2**	10.3	10.6	12.7	21.5	**19.2**	19.3	22.2
Qwen2.5-0.5B	PPL	**14.31**	14.63	14.77	15.07	**21.84**	21.91	22.18	22.39
(dense 13.07)	KL	**0.098**	0.122	0.129	0.148	**0.514**	0.523	0.541	0.550
	Flips (%)	**11.4**	12.0	12.7	12.8	21.8	**21.3**	21.4	21.4

Table 1: Equal storage, mask counted: WikiText-2 perplexity, KL to the dense model (nats/token), flips (% of items, nine tasks); seed 0; bold: best.

5 Results

Storage: pruning pays only when it replaces a bit. Table 1 and Fig. 1 compare configurations on the storage axis. At 4.0 bits per weight, 25% pruning brings nothing over plain 4-bit: on SmolLM2-135M its best order, P→Q, reaches a perplexity of 21.99 against 21.94, and on every model the best order is within 2.2% of 4-bit. At 3.0 bits, 50%+4-bit beats plain 3-bit: on SmolLM2-135M, P→Q reaches 41.20 against 57.55, with lower KL (0.966 against 1.273) and fewer flips (22.0% against 27.4%), and its perplexity is lower on 2 of 3 models, by 6.8–28.4%; on Qwen2.5-0.5B the two tie (21.91 against 21.84). TCr would put 25%+4-bit at 3.0 bits, where it seems to beat 3-bit by far, as in [4]. An entropy-coded mask leaves the 3.0-bit comparison unchanged ($H_2(0.5) = 1$) and makes 25%+4-bit 0.19 bit cheaper than 4-bit at about equal quality. Under symbol entropy, which also frees the zeros of plain rounding, 3-bit costs 2.42 bits per weight on SmolLM2-135M and 50%+4-bit 2.62: that comparison is no longer at equal storage.

Free zeros. Rounding creates zeros that TCr ignores: 3-bit quantization sets 30.7% of the weights of SmolLM2-135M exactly to zero (4-bit: 14.8%). A grid-first 25% mask then picks only weights that are already zero: Q→P at 25%+3-bit is the 3-bit model, with the same perplexity and layer errors, on 3 of 3 models. TCr credits it with 2.25 bits per weight and the bitmap with 3.25; it is a 3.0-bit model with free sparsity.

Sub-additivity with one Hessian. With one shared Hessian per layer (E2), mask-first orders are sub-additive in aggregate in 36 of 36 combinations of model, cell and order. Per layer, the joint sweep is sub-additive in 97–100% of the layers of a cell, P→Q only in 60–92%. Its interaction grows with input correlation, as the cross terms allow: the per-layer ratio has a Spearman correlation of +0.41 to +0.60 ($p \leq 5.43 \times 10^{-6}$) with the mean absolute correlation between the layer's inputs. In block 0, P→Q is super-additive in 6, 1 and 0 of

the six cells on SmolLM2-360M, -135M and Qwen2.5-0.5B.

Order: it matters, but no proposed rule or mechanism explains it. Joint has the lowest WikiText-2 perplexity in 11 of 18 cells, P→Q in 7, Q→P in 0; Q→P is the worst in 12 of 12 4-bit cells and reaches 1.77 times the best (SmolLM2-135M 50%+3). Kim et al.'s weaker-first rule [7], with their compression-equivalent ratio, picks the better sequential order in 13 of 18 cells, but in 0 of the 4 where pruning is the stronger step and it predicts quantizing first. Sparsity first wins in 17 of 18, as Harma et al. prove for magnitude pruning [6], yet the reverse order costs up to 77.3% in perplexity. The quantize-first penalty has two proposed causes: a stale grid, fitted before pruning (our earlier reading), or a mask chosen on rounded weights (as the analysis of [6] suggests). Four arms separate them on one Hessian (E2; per-cell values in the repository): Q→P with the grid re-fitted on the kept weights, with the mask of pruning alone, or both, and P→Q on a stale grid. Re-fitting increases the penalty in 12 of the 12 cells that have one; the pruning-only mask removes at most 39% of it, and a negative share in some cells; the stale grid changes the error of P→Q by at most 0.4%. Neither reading explains the penalty; its cause remains open. In full runs (E3, 50%+4-bit, SmolLM2-135M), the arms remove −35%, +9% and +39% of the perplexity penalty.

Evaluation. C4 perplexity and KL rank the orders as WikiText-2 perplexity does in 16 and 17 of 18 cells, flips and accuracy in only 9 and 10. Flips show what retained accuracy hides: 4-bit SmolLM2-135M keeps 0.942 of the dense accuracy (Sr [4], three tasks) but flips 14.4% of its answers. McNemar tests separate P→Q from Q→P in 17 of 21 cells ($p < 0.05$), not always as perplexity does: at 50%+4-bit on SmolLM2-135M, P→Q has the lowest perplexity and KL, yet Q→P answers more items correctly, mostly on SciQ (mean accuracy over the nine tasks, P→Q minus Q→P: −1.4 points, 95% interval −2.1 to −0.7).

Noise. Within one GPU type results are bit-exact (two L4 machines give the same likelihood and all 210 layer errors for SmolLM2-135M at 50%+4-bit), and fp32 reproduces our fp32 T4 runs of SmolLM2-360M within 0.4%. Across GPU types, the same 16-bit code changes SmolLM2-135M perplexity by +1.5% at 4 bits and +4.6% at 50%+4-bit between a T4 and an L4, presumably because flipped rounding and mask decisions cascade through the sequential calibration. Gaps of a few percent between orders are thus weak evidence on their own. Over three calibration seeds of SmolLM2-360M, the best order is the same in every seed in 3 of 3 4-bit cells, joint beats P→Q in every seed in 3 of 3 (paired t-test $p < 0.05$ in 3), and the standard deviation of perplexity reaches 0.2% for single methods and 1.0% for combinations. With three models of two families, we claim no scale trend.

6 Conclusion

Inside one Hessian-compensated pipeline, quantizing first is the worst order in most settings, but neither a stale grid nor a mask chosen on rounded weights explains why, and mask-first orders are sub-additive in aggregate, not in every

layer. Counted honestly, pruning pays only when it replaces a bit of precision: 50% pruning with 4-bit weights beats plain 3-bit at 3.0 bits per weight on 2 of 3 models, whereas 25% pruning with 4-bit weights only ties plain 4-bit at 4.0 bits, and the zeros that rounding creates count for plain quantization too. We recommend reporting bits per weight with the mask, KL divergence, flips and paired tests.

References

[1] Elias Frantar and Dan Alistarh. SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *PMLR*, pages 10323–10337, 2023.

[2] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. OPTQ: Accurate quantization for generative pre-trained transformers. In *International Conference on Learning Representations (ICLR)*, 2023. Preprint title: GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers, arXiv:2210.17323.

[3] Elias Frantar, Sidak Pal Singh, and Dan Alistarh. Optimal brain compression: A framework for accurate post-training quantization and pruning. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Stanislas Laborde, Martin Cousseau, Antoun Yaacoub, and Lionel Prevost. Semantic retention and extreme compression in LLMs: Can we have both? In *2025 International Joint Conference on Neural Networks (IJCNN)*, pages 1–9. IEEE, 2025.

[5] Zihan Zhou, Simon Kurz, and Zhixue Zhao. Revisiting pruning vs quantization for small language models. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 12055–12070, 2025.

[6] Simla Burcu Harma, Ayan Chakraborty, Elizaveta Kostenok, Danila Mishin, Dongho Ha, Babak Falsafi, Martin Jaggi, Ming Liu, Yunho Oh, Suvinay Subramanian, and Amir Yazdanbakhsh. Effective interplay between sparsity and quantization: From theory to practice. In *International Conference on Learning Representations (ICLR)*, 2025.

[7] Minjun Kim, Jaehyeon Choi, Hyunwoo Yang, Jongjin Kim, Jinho Song, and U Kang. Prune-then-quantize or quantize-then-prune? Understanding the impact of compression order in joint model compression. In *International Conference on Learning Representations (ICLR)*, 2026.

[8] Iheb Bouriel, Qassim Nasir, and Manar Abu Talib. Interaction effects in hybrid compression of small language models. In *Proceedings of the Thirty-Fifth International Joint Conference on Artificial Intelligence, IJCAI-26*, pages 5666–5674, 2026.

[9] Hang Guo, Luca Benini, and Yawei Li. Optimal brain restoration for joint quantization and sparsification of LLMs. In *International Conference on Learning Representations (ICLR)*, 2026.

[10] Loubna Ben Allal et al. SmolLM2: When smol goes big — data-centric training of a fully open small language model. In *Conference on Language Modeling (COLM)*, 2025.

[11] Qwen Team. Qwen2.5 technical report, 2024. arXiv:2412.15115.

[12] Abhinav Dutta, Sanjeev Krishnan, Nipun Kwatra, and Ramachandran Ramjee. Accuracy is not all you need. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2024.

[13] Colin Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

[14] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017.

[15] Leo Gao et al. The language model evaluation harness. Zenodo, 2026. Version v0.4.13, doi:10.5281/zenodo.22214970.